
MEASURING AI CODING AGENT USE FROM REPOSITORY TRACES: EVIDENCE FROM A HACKATHON WHERE CODEX WAS REQUIRED

PREPRINT

Talgar Bayan^{1,2}

¹School of Computing and Artificial Intelligence (SCAI), Nazarbayev University, Astana, Kazakhstan

²University of Manchester, Manchester, United Kingdom

talgar.bayan@gmail.com

27 September 2026

ABSTRACT

AI coding agents are increasingly used to write software, and their adoption is often measured from the traces they leave in repositories. Such measurements rarely know which repositories could have shown activity or which tool developers were meant to use. We study HackAlem AI, a five-hour hackathon in Astana on 23 September 2026, where every team had to use OpenAI Codex and the organisers created a public GitHub repository for registered teams in advance. We mirrored all 3,649 repositories with full history, derived measures with deterministic rules and reproduced the core counts independently. Organiser actions shaped the population: 446 repositories were archived before the event and an evening batch of 1,040 stayed almost unused, so event-time work reached 29.0% of all repositories but 48.2% of the rest. Codex left a detectable trace in 46.4% of active repositories, and in 69.6% of those whose README named it; its commits were seldom signed, whereas Claude Code, which was not required and signs commits by default, appeared in 24.9%. Strings matching OpenAI key formats reached the public history of 61 repositories, mostly through environment files. Repository-based measures of agent use therefore depend on how the population was formed and on tool defaults.

Keywords AI coding agents · agentic software engineering · mining software repositories · hackathons · OpenAI Codex · credential leakage

1 Introduction

AI coding agents such as OpenAI Codex and Claude Code edit whole repositories from natural-language instructions, and in many real sessions the agent writes nearly all of the committed code [1]. How widely such agents are used, and with what effect, is increasingly measured from the traces they leave in version control: co-author trailers in commit messages, branch names, pull requests and context files such as `AGENTS.md` [2–5]. Large studies built on these traces report adoption rates, productivity gains and quality costs [3, 6, 7]. Their conclusions depend on how completely the traces record agent use.

Two quantities are usually unknown in this kind of research. The first is the denominator: the set of repositories that could have shown activity, including those that were never used. Samples drawn from project listings or from detected agent activity cannot contain them [8, 9]. The second is the tool the developers were meant to use. An agent that does not sign its commits leaves no signature, so a repository without a trace may belong to a team that used an agent silently, and no study can tell how many such repositories it misses [4].

Hackathons offer a setting in which both quantities can be fixed in advance. They are time-bounded events at which small teams build prototypes [10], and their repositories have been studied at scale, mostly from projects that teams chose to list on platforms such as Devpost [11–15]. Studies of generative AI at hackathons have so far relied on

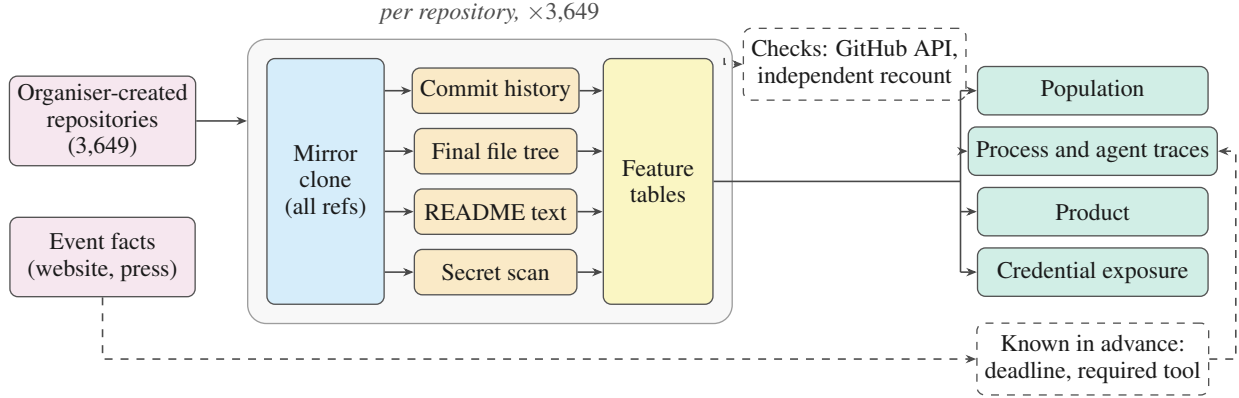


Figure 1: Study architecture. Every organiser-created repository was mirrored with all its references and passed through the same pipeline into feature tables, which were checked against the GitHub API and an independent recount. Four analyses use these tables. The event facts fix the deadline and the required tool, against which the traces are compared.

interviews, surveys or single cases [16–19]. HackAlem AI, held in Astana on 23 September 2026, differed in two respects. Before the event, the organisers created a public GitHub repository for registered teams, so the organisation holds repositories that never received a commit as well as those that did. The rules also required every team to use Codex [20], so the tool the traces should reveal is known.

We mirrored all 3,649 repositories with their full history and ask four research questions. The first (RQ1) asks how many repositories carried work during the event and how much of the gap reflects the organisers’ own actions. The second (RQ2) asks how teams spread their work over the five hours and which traces the required and a non-required agent leave, measured against what teams wrote in their READMEs. The third (RQ3) asks what teams built and documented, and the fourth (RQ4) how credentials reached the public history. Figure 1 shows the study design.

The organisers’ actions account for most of the gap between repositories and activity: 446 repositories were archived before the event, and a batch of 1,040 created the evening before stayed almost unused. Codex, the required tool, was detected in 46.4% of active repositories. Even among the repositories whose README named Codex, its traces appeared in only 69.6%, and Codex commits were seldom signed. Claude Code, which signs commits by default [21], was detected in 24.9%. Keys reached the public history mainly through committed environment files, not through agent-signed commits.

The paper makes four contributions. First, it reports participation on a known denominator and separates the organisers’ actions from the teams’ activity, which studies of organiser-created repositories have to do before any rate can be read. Second, it measures what the trace heuristics used in agent-adoption studies recover when the intended tool is known, using README self-reports as an independent reference. Third, it gives population-level estimates, with intervals, of the artefacts, documentation and exposed credentials of agent-assisted prototypes built under one deadline. Fourth, it releases a pseudonymised dataset and a pipeline in which every number is generated from the data, the core counts are reproduced by independent code, and the headline results can be recomputed from the public files alone.

Section 2 reviews related work and Section 3 describes the event. Section 4 sets out the data, measures, threat model and statistics, and Section 5 presents the results. Section 6 discusses them, Section 7 examines threats to validity, and Section 8 concludes.

2 Background and related work

2.1 Hackathons and their repositories

Hackathon research spans the purpose, format, processes and outcomes of events [10]. Repository studies have related project continuation to preparation, winning and team skills [11], found that most commits of hackathon projects fall in the first week and that few projects remain active months later [12], and measured how much code is written during an event and reused afterwards, starting from 22,183 projects listed on Devpost [13, 14]. HackRep collects 100,356 hackathon repositories and shows that event durations can be estimated from commit timestamps [15]. Samples of listed projects cannot contain registrations that produced no code. General cautions for mining GitHub apply to all of these data sets: many repositories are inactive or personal [8], git histories can be rewritten and their timestamps are set

by clients [22], mining tools disagree on basic counts [23], and representative samples are rare in software engineering research [9].

2.2 Measuring coding-agent use from repositories

Before agents, ChatGPT use in open-source projects was found by matching messages that mention it and checking each match by hand [24]. Agents leave more explicit traces. The AIDev dataset collects pull requests authored by five agents [2], and studies of projects that adopted agents report velocity gains, largest when the agent is the first AI tool a project uses, together with lasting increases in code complexity and static-analysis warnings [6, 7]. Robbes et al. estimate that 22.20–28.66% of 128,018 GitHub projects had adopted a coding agent by February 2026, find that commits assisted by agents are larger than commits written by developers alone, and describe the heuristics and pitfalls of detecting agents from traces [3, 4]. Agent context files have been characterised in open-source projects [5]; Codex reads AGENTS.md before doing any work [25], and in a benchmark such files lowered task success on average [26]. Agents are also evaluated on repository tasks [27, 28]. These studies observe agents in projects whose developers chose their tools freely, so the share of agent use that the traces miss cannot be estimated from them.

2.3 Generative AI in hackathons and programming education

In a controlled experiment, developers with GitHub Copilot finished a task faster than a control group [29], and in a field study perceived productivity was linked to how often suggestions were accepted [30]. Observational work describes separate acceleration and exploration modes of interaction [31], and developers surveyed report both benefits and trouble controlling the tools [32]. At hackathons, participants use generative AI for coding, learning and documentation and check its output under time pressure [16]. Novices can deliver working demonstrations while engaging little with engineering practice [17], a rule against manual editing shapes prompting and debugging [18], and a university case discusses educational opportunities and risks [19]. Building software mainly by prompting, often called *vibe coding*, has been described as a trade between speed and technical debt [33], and computing educators have reviewed what these tools mean for teaching [34, 35]. On the use of time, commit timestamps show when developers work [36], students who start earlier tend to finish earlier and score higher [37, 38], most studies of time pressure report higher productivity and lower quality [39], and developers describe procrastination and its causes in interviews [40].

2.4 Credentials in public repositories

Secrets leak to public GitHub repositories often and remain there; a six-month scan of public commits in real time found thousands of new secrets every day [41]. Detection methods have been proposed [42], a labelled dataset supports the evaluation of detectors [43], and a comparison of detectors found Gitleaks to have the highest recall (88%) with lower precision (46%) [44]. Developers who leaked secrets describe remediation as hard [45]. Code completion models can reproduce credentials seen in training [46], and participants with an AI assistant wrote less secure code while believing it more secure [47]. For AI-built software, LLM API keys leak from mobile apps [48], and agent-generated and *vibe-coded* applications contain vulnerabilities [49, 50].

2.5 Research gap

Repository studies of agents know neither their denominator nor the tool developers were meant to use, and studies of AI at hackathons rely on interviews, surveys or single cases rather than on the output of a whole event. We found no repository-level study of a hackathon in which repositories were created for registered teams in advance and one agent was required, which is the setting that allows trace-based measures of agent use to be checked against a known intended tool.

3 Study setting

Table 1 lists the facts about the event that we use, with their sources. Registration was free and people could register alone [20]. News reports describe long queues at the entrance and registered participants who could not get in; the organisers said that participants had been told in advance that seats were limited [51, 52]. The event had 12 tracks, each set by an industry or public-sector partner (Table 3).

Each repository was named `hack-<id>-<team>` and began with an organiser commit titled “Initial commit” that added a README of two lines. The organisers archived the repositories, which made them read-only: 3,201 were archived between 18:01 and 18:51 on the event day, all 3,649 were archived when we collected the data, and Section 5.1 shows that the rest were archived before the event began.

Table 1: Facts about the event. Organiser and news sources were read on 24 September 2026 or later; the event window is our inference.

Item	Value	Source
Date and place	23 September 2026, Astana, in person	[20]
Duration	five hours	[52–54]
Event window	13:00–18:00 local time (UTC+5)	this study
Teams	up to three members; individual registration allowed	[20]
Tooling rule	teams must use Codex during development	[20]
Capacity	2,500 seats; 6,586 applications	[20, 51, 52]
Participants	more than 2,500 from 21 countries and 129 universities	[53]
Record attempt	Guinness World Records, most participants in an agentic AI hackathon; previous record 2,089; result pending	[53]
After the event	jury review of submitted projects 24–28 September; finalists present and winners are announced on 29 September; awards on 1 October 2026	[20]
Repositories	public repositories in BAITC-Hacks, one per registered team	this study

4 Methodology

4.1 Data collection

On 24 September 2026 we listed all repositories of the organisation through the GitHub REST API, including their creation and last-update times, collected default-branch commit counts through GraphQL, and made mirror clones of all 3,649 repositories. The clones keep every branch, every pull-request reference and the full history. All clones completed, and because the repositories were archived their content could no longer change. All measures below are computed by deterministic scripts from the clones; no language model labels or classifies the data.

4.2 Units of analysis and definitions

A *team commit* is a commit reachable from any branch, counted once per SHA, other than the organisers’ template commit. We identify the template commit by a rule that does not depend on the author account: a commit without parents, titled exactly “Initial commit”, made within 120 s of the repository’s creation, that adds two lines to one file and deletes none. The *event window* is 13:00–18:00 local time on 23 September 2026, measured on committer timestamps; we inferred it from commit and archive times together with press reports of a five-hour event, since we found no official schedule. An *active repository* has at least one team commit in the window. For each active repository i with c_i window commits, of which ℓ_i fall between 17:00 and 18:00, the *last-hour share* is $s_i = \ell_i/c_i$; an even spread of work over the five hours would give 20%.

We read a last-update time before the start of the window as *archived before the event*, because archiving updates this time and none of these repositories received a commit during the event. The *evening batch* consists of the repositories created between 18:00–20:00 on 22 September. We treat it as an organiser action, because up to 15 repositories were created per minute then, against at most 8 in any minute of earlier days. A repository stands for one registration of up to three people, and its creation date stands in for the registration date; we make no claims about individuals. The population analysis uses all 3,649 repositories. All other analyses use the 1,057 active repositories; README measures use the 1,033 of them with a written README, and commit measures use their team commits in the window unless stated otherwise.

4.3 Measures

From the final state of each default branch we record the primary language by file size, frameworks from package manifests, and LLM use from manifests, import statements and API endpoints, excluding vendored folders. Engineering artefacts are test files (identified by file and folder names), a Dockerfile, a Compose file, a CI workflow and a deployment configuration.

For each agent we record three kinds of repository trace. Context files are AGENTS.md at any depth, which Codex reads [25] and other agents read too, CLAUDE.md or a .claude folder for Claude Code, and the rule files of Cursor, Copilot and Gemini. Commit signatures are co-author trailers or “generated with” lines that name an agent, and author or committer accounts that are agent accounts or carry exactly a tool’s name; generic bot accounts, among them the organisers’ application, are not counted. Branch traces are branch names that start with an agent’s name. Pull-request references are reported separately as a workflow trace, because teams create them with or without agents. As a fourth

Table 2: Validation and sensitivity checks.

Check	Result
Default-branch commit counts, clones vs. GitHub API	3,649 of 3,649 equal
Independent recount of team commits / window commits / hours with commits	3,649 / 3,649 / 3,649 of 3,649 equal
Template commit found by the account-independent rule	3,615; 34 rewrote their history
Template commits by a second organiser account	16 of 402 audited
Consistency with the earlier table (team / window commits, hours)	3,641 / 3,644 / 3,648 of 3,649 equal
Active repositories with author time instead of committer time	1,057 (committer time: 1,057)
Active repositories with the window shifted by $-60 / -30 / +30 / +60$ min	1,026 / 1,040 / 1,060 / 1,059
Track labels vs. keyword rules (active, one clear track)	933 of 946 agree; $\kappa = 0.98$
README language rule vs. blind hand labels (81 READMEs)	[pending: labelling in progress]

channel, independent of the traces, we record whether the README names Codex or Claude, which we treat as a self-report of use.

For the README at the repository root we remove code, links and markup and classify the remaining prose by its letters. A README is Kazakh if at least 50% of its letters are Cyrillic and at least 3% of those are among the nine letters that Kazakh has and Russian does not; Russian if at least 50% of its letters are Cyrillic but it does not meet the Kazakh condition; English if at least 80% of its letters are Latin; and mixed otherwise. The script of commit subject lines is classified in the same way. README content has been categorised in earlier work [55]; we record the length of the README, whether it has run or setup instructions (a heading on installation, setup or usage, or a command inside a code block), images, and a link to a deployed application. For each non-merge commit we count the lines changed outside lock files, data files and generated output, so that the size of a commit reflects the code and text that were written.

4.4 Credential scanning and threat model

We ran Gitleaks [56] version 8.30.1 with redacted output over every reference of the 1,254 clones with team commits. For each finding we kept, privately, the rule, file, commit, line, date and whether the matched string was still present in the final state. We never stored or printed a matched value and never tested one against a service. For provider-specific rules, a string counts as a placeholder if it contains placeholder text (such as a run of x, “your”, “example” or angle brackets) or has a character entropy below 3.5 bits.

The threat model is that of an outside observer who can read every public repository and its full history, as a real-time scan of public commits can [41], but has no other access. The assets are credentials for paid services used during the event, mainly LLM API keys. A credential counts as exposed if a string in its provider’s key format appears in any commit reachable from a public reference, whether or not it remains in the final version, because the history of an archived repository stays readable. We do not know whether a key was valid, revoked or used, since testing keys would itself be misuse. Our counts are therefore upper bounds on usable keys and lower bounds on exposure, because keys in formats that the rules do not cover are missed [44].

4.5 Track labels

Track labels come from an earlier per-repository table that the author produced for a public report on the event. We checked them against README text with keyword rules. Where the keyword rules gave one clear track, they agreed with the label for 933 of 946 active repositories (98.6%, $\kappa = 0.98$). We read every disagreement and corrected 7 labels. Of the labelled repositories, 19 had no README text to check, and a further 73 active repositories stayed unassigned.

4.6 Validation

Table 2 lists the checks. Default-branch commit counts from the clones equal those from the GitHub API for every repository, and an independent script that shares no code with the pipeline reproduced team commits, window commits and hours with commits for every repository. While spot-checking repositories on GitHub we found template commits made by a second organiser account; an audit of all 402 repositories this could affect found 16 such repositories, which the account-independent rule now handles. Commits reachable only through pull-request references (427 commits in 117 repositories) are not team commits. The number of active repositories hardly changes when the window is shifted or author time replaces committer time.

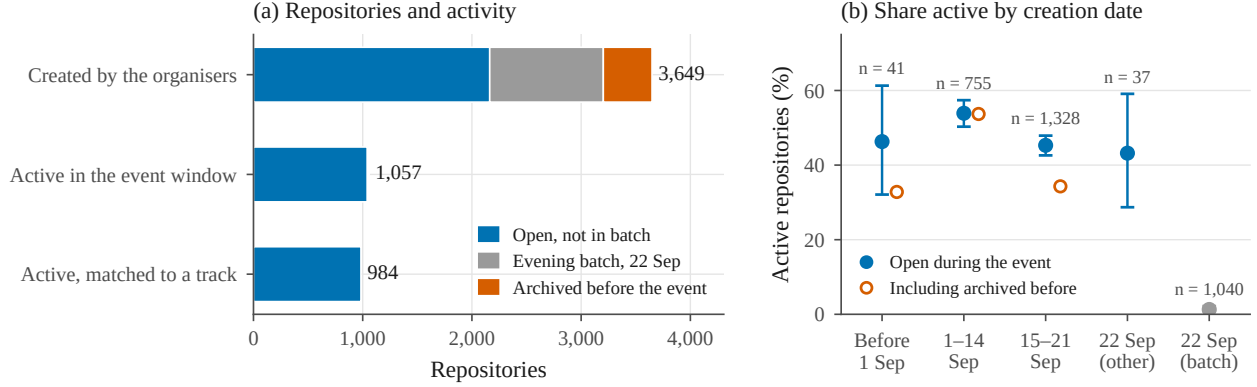


Figure 2: Population. (a) Repositories created by the organisers, split into those open during the event and not in the evening batch, the batch, and those archived before the event; below them, the active repositories and those matched to a track. (b) Share of repositories that became active by creation date, with 95% Wilson intervals, for repositories open during the event (filled) and for all repositories, including those archived before the event (open circles).

4.7 Statistical analysis

The repositories are the complete set for this event, so intervals and tests describe how precisely the observed shares reflect the process that produced them, not sampling from other events [9]. Proportions carry Wilson 95% confidence intervals, and distributions are summarised by the median and interquartile range (IQR). Groups are compared with χ^2 tests, with Cramér’s V as the effect size and the number of cells with small expected counts reported, and with Mann–Whitney tests, with Cliff’s δ . One association is adjusted with logistic regression and one with a Mantel–Haenszel odds ratio. Tracks are compared with a Kruskal–Wallis test, with $\varepsilon^2 = H/(n - 1)$ as the effect size. The sizes of agent-signed and other commits are compared within repositories that have both, using each repository’s median for each kind of commit and a Wilcoxon signed-rank test. The three tests about AGENTS.md form one family with Holm correction, and the other tests stand alone. We fixed the tests after exploring descriptive numbers and before running them. The plan, the deviations from it and their reasons are released with the data, and analyses added after the plan are labelled *exploratory*. Every number in the paper is generated from the data by the released scripts.

5 Results

5.1 Population and participation

The organisers created 3,649 repositories, but not all of them were open during the event (Figure 2a). Of these, 446 (12.2%) had been archived before it began, in three bulk operations: 126 repositories between 18:30 and 18:32 on 15 September, 56 on 17 September, and 242 between 20:01 and 20:05 on 22 September, with 22 more on other days. Most of them (426) had been created between 15 September and 21 September, and the last operation came minutes after the evening batch of 1,040 new repositories was created. None of the repositories archived before the event became active, although 157 of them already held team commits from before it. Of the batch, 14 (1.3%) became active.

Over all repositories, 1,254 (34.4%) have team commits and 1,057 (29.0%, 95% CI 27.5–30.5) are active. Among the 3,203 repositories open during the event the active share is 33.0%, and among those that were open and not in the batch it is 48.2% (1,043 of 2,163; 95% CI 46.1–50.3). Within this last group, activity varied little with the creation date (Figure 2b), from 43.2% for repositories created on 22 September to 53.9% for those created between 1 September and 14 September ($\chi^2(3) = 14.9$, $p = 0.002$, $V = 0.08$). Among all repositories created between 15 September and 21 September the share was lower (34.3%, against 45.3% of the open ones), and the archiving accounts for the difference.

Of the 197 repositories with team commits but none in the window, 194 had all of them before the event and 157 were archived before it. The active repositories hold at most 2,740 distinct author identities (median 3 per repository). This is an upper bound on the number of people, because one person can commit under several e-mail addresses; press reports give more than 2,500 participants [53]. Of the active repositories, 984 (93.1%) could be assigned to a track. In short, most of the distance between the repositories the organisers created and the repositories that received work during the event comes from repositories that were closed or barely used before the event started.

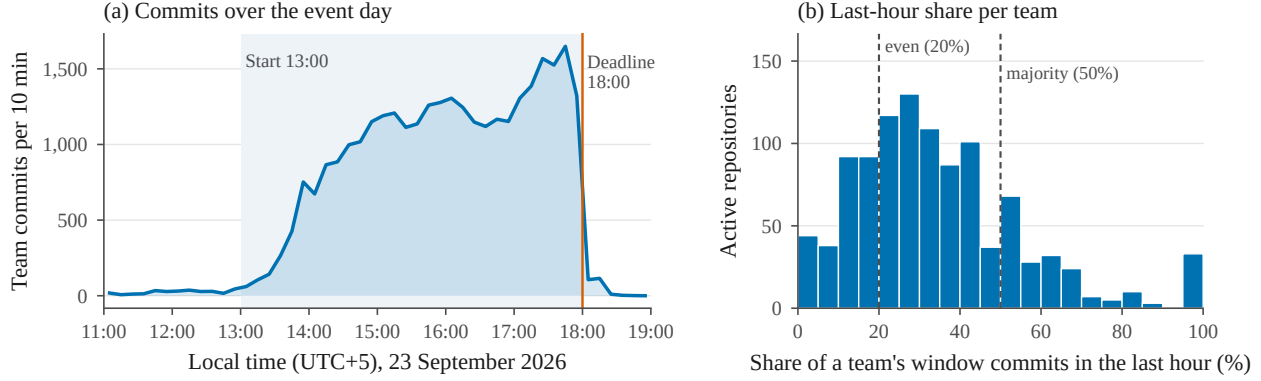


Figure 3: Work over the event day. (a) Team commits per ten minutes, all branches of all repositories; the shaded band is the event window. (b) Distribution over the active repositories of the last-hour share s_i . Dashed lines mark an even spread and a majority.

5.2 Work over the event window

Active repositories received 30,418 team commits in the window. Activity rose during the first hour and stayed high through the afternoon, and the last hour held 28.8% of all window commits; the busiest ten minutes were 17:40–17:50, with 1,649 commits (Figure 3a). The median last-hour share of a team was 30.8% (IQR 19.4–43.8%), above the 20% of an even spread (Figure 3b), and 163 teams (15.4%, 95% CI 13.4–17.7) made most of their commits in that hour. At the same time, 79.3% of teams committed in at least four of the five hours and 44.5% in all five. The median team made its first window commit 61 minutes after the start (IQR 43–94), its last 9 minutes before the deadline (IQR 3–20), and 19 commits in all (IQR 10–35). Some active repositories also had commits from before the event (173, of which 33 changed more than a thousand lines) or after the deadline (73).

In an exploratory analysis, clustering the teams’ hourly profiles separated a late group of 323 teams, but the structure was weak: the silhouette was 0.30, against 0.19 for simulated data with no team structure [57]. The teams form a continuum rather than types. Most worked through the afternoon with a moderate rise towards the deadline, and a minority left most of their commits to the last hour.

5.3 Traces of coding agents

Codex, the required tool, left at least one repository trace in 490 active repositories (46.4%, 95% CI 43.4–49.4; Figure 4a). An AGENTS.md file appeared in 33.3%, a branch starting with codex in 18.9% and a Codex signature in 4.4%. Claude Code, which was not required, left a trace in 24.9%, mostly through signed commits (16.9%) and CLAUDE.md files (14.2%). The two tools overlapped: 309 repositories showed only Codex, 181 both, 82 only Claude Code and 485 neither. Other agents appeared in few repositories (Cursor 20, Copilot 5, Gemini 2). Of all team commits, 3,953 (12.4%) carry an agent signature, 3,503 of them naming Claude and 332 naming Codex. Pull-request references, a workflow trace, occur in 35.9% of active repositories, 143 of them together with a Codex branch.

The README self-reports, an exploratory addition, show how much the traces miss (Figure 4b). The README named Codex in 168 active repositories (15.9%), and only 69.6% of them (95% CI 62.3–76.1) carried a Codex trace: an AGENTS.md file in 49.4%, a codex branch in 30.4% and a signature in 7.7%. The README named Claude in 81 repositories, of which 77.8% carried a Claude Code trace and 58.0% a signature. Among repositories that named the tool, signatures therefore revealed Claude Code in more than half of the cases and Codex in fewer than one in ten. Counting the README mention as evidence raises the share of active repositories with evidence of Codex from 46.4% to 51.2%.

Agent-signed commits differed from the others in form. In the 217 active repositories that had both kinds, the median signed commit changed 240 lines and the median other commit 202, excluding lock files, data and generated output. Signed commits were larger in 60.4% of these repositories (95% CI 53.7–66.6; Wilcoxon $p = 0.002$). Signed commits also more often had conventional-commit subjects (48.5% against 42.1% of non-merge commits). In a second exploratory comparison, repositories with an AGENTS.md file had more window commits (median 28 vs. 16; $\delta = 0.36$, Holm-adjusted $p < 0.001$), more hours with commits (median 5 vs. 4; $\delta = 0.26$, $p < 0.001$) and more often test files (95.7% vs. 82.7%, $p < 0.001$). The difference in test files remains after adjusting for the number of window commits (odds ratio 3.51, 95% CI 1.99–6.17). Teams that work through agents may produce both the file and the tests, and teams that prepared more may differ in other ways, so these are associations only.

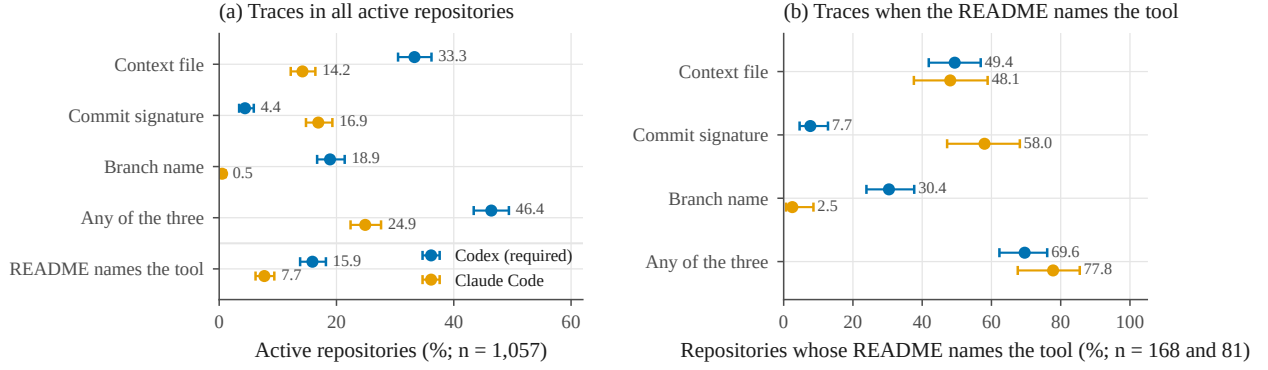


Figure 4: Traces of coding agents. (a) Share of active repositories with each kind of trace, and with a README that names the tool, for Codex, which every team had to use, and for Claude Code. (b) The same traces among the repositories whose README names the tool. Points are shares with 95% Wilson intervals.

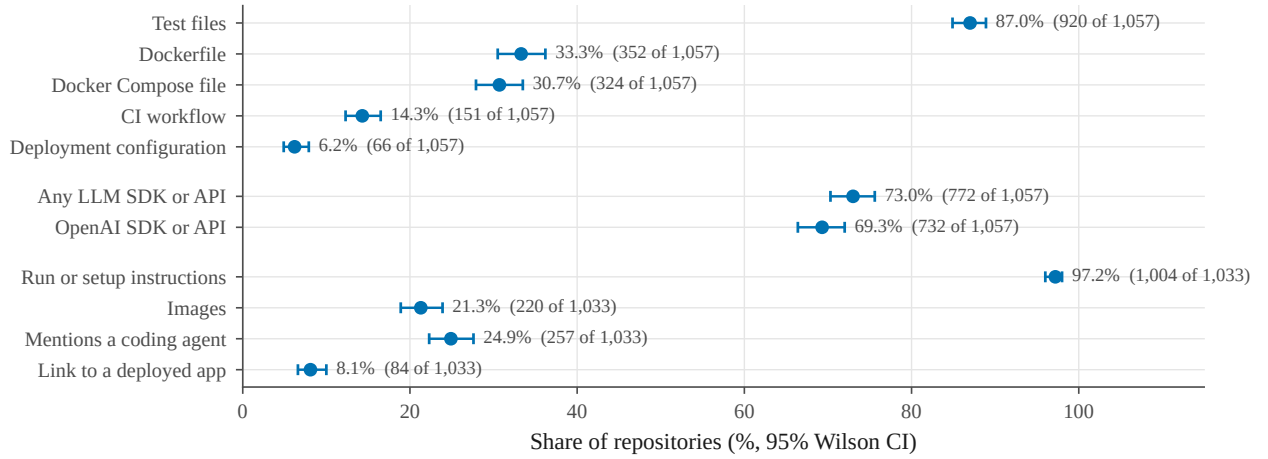


Figure 5: What teams built. Engineering artefacts and LLM use in the 1,057 active repositories, and README features in the 1,033 of them with a written README; shares with 95% Wilson intervals.

5.4 What teams built and documented

Python was the primary language of 58.5% of active repositories, followed by TypeScript (21.7%) and JavaScript (12.5%). The most common frameworks were React (46.5%), FastAPI (43.2%), Vite (32.8%), Tailwind (21.2%) and Next.js (16.5%). An LLM SDK or API call appeared in 73.0% of repositories, OpenAI’s in 69.3%, and each other provider in at most 2.9%. Test files were present in 87.0% of repositories, a Dockerfile in 33.3%, a CI workflow in 14.3% and a deployment configuration in 6.2% (Figure 5). These shares hardly change when repositories with commits before the event or after the deadline are left out (test files 86.6% of 831). We did not run the tests, so their presence shows an intention to test, not working tests.

Of the 1,033 active repositories with a written README, 88.1% were in Russian, 9.3% in English, 2.1% in Kazakh and 0.5% mixed; Kazakh-specific letters appeared somewhere in 14.2%. Commit subjects, by contrast, were mostly in Latin script (87.5% of non-merge window commits, against 12.1% in Cyrillic). READMEs were long, with a median of 1,645 words (IQR 1,049–2,413) and 16 headings. Of these READMEs, 97.2% gave run or setup instructions, 21.3% included images and 8.1% linked to a deployed application.

The primary language differed between tracks (Table 3; $\chi^2(33) = 201.1$, $p < 0.001$, $V = 0.26$, with 8 of 48 cells having expected counts below five). Python ranged from 30.4% of repositories in the city-simulator track to 95.7% in the telecom track. Window commits per team varied little between tracks (medians 16–31; $H(11) = 19.0$, $p = 0.062$, $\epsilon^2 = 0.02$). Kazakh READMEs were rare in every track, including the one whose task was to take minutes from Kazakh, Russian or mixed meeting audio (4.6%). Within five hours, then, most teams produced a Python or TypeScript application that called an LLM, with test files and a long README, mostly in Russian.

Table 3: The 12 tracks with the active repositories matched to each, their median number of window commits, and the shares with test files, with Python as primary language and with a Kazakh README (of written READMEs). Partners are named in team READMEs; task descriptions are our summaries of those READMEs.

	Track	Partner	Task	Repos	Commits (median)	Tests (%)	Python (%)	Kazakh (%)
T1	Energy	Samruk-Kazyna	Agent that forecasts wind-farm output from weather data	74	25	90.5	83.8	2.7
T2	Finance	Freedom	Rebuild an organised group from a transaction network	115	21	87.0	74.8	1.8
T3	Management	Halyk Bank	Guide linking employee skills to growth steps	89	21	93.3	60.7	1.1
T4	Telecom	Beeline	Agent choosing tariff offers for subscribers	46	18	76.1	95.7	0.0
T5	Logistics	ekt.kz	Supplier orders to replenish stock, approved by a manager	80	22	91.2	67.5	1.2
T6	Creative industries	Firebird	Choose event contractors and explain the choice	113	16	88.5	63.7	1.8
T7	Education	AI Sana	Turn a business problem into a task for student teams	145	20	86.2	31.0	1.4
T8	Innovation	Samruk-Kazyna	Minutes and action items from Kazakh, Russian or mixed meeting audio	65	17	89.2	87.7	4.6
T9	Communications	Halyk Bank	Voice agent that routes calls in an insurance contact centre	29	31	79.3	72.4	0.0
T10	Trade	ekt.kz	Shopping assistant: products, alternatives, prices, stock	71	19	90.1	49.3	5.6
T11	Special	Kazakhtelecom	Compare organisation charts before and after a reorganisation	42	20	92.9	61.9	2.4
T12	Special	Astana Innovations	City simulator that splits a budget across districts	115	23	94.8	30.4	0.9

5.5 Credential exposure

Gitleaks’ generic rule produced 4,597 findings in 131 repositories, 93.5% of them in data files (`.json`, `.jsonl`, `.txt` or `.csv`); we treat these as low-precision and do not analyse them further. Provider-specific rules produced 127 findings, of which 10 were placeholders. The other 117 lie in 73 active repositories (6.9%), and in 36 of them (3.4%) the string is still present in the final version (Figure 6a). Most are OpenAI API keys: 77 findings in 61 repositories (5.8%, 95% CI 4.5–7.3), of which 30 still hold the string. By the commit dates that Gitleaks reports, all were committed during the event window.

The OpenAI key strings sat mainly in environment files (Figure 6b): 32 findings in `.env.example` and 27 in `.env`. A `.env.example` file is a template meant to be committed, so a real key placed there becomes public even when `.gitignore` excludes `.env`. `.gitignore` covered `.env` in 83.6% of active repositories, a `.env.example` file was present in 74.3%, and 3.2% committed a `.env` file. In an exploratory analysis, key strings were not clearly more frequent where `.gitignore` covered `.env` (6.1% vs. 4.0%, $p = 0.287$). Coverage of `.env` went with LLM use (90.9% of repositories with an LLM SDK against 63.9% without), and adjusting for LLM use gave a Mantel–Haenszel odds ratio of 1.03 (95% CI 0.45–2.37). The key strings were also not tied to agents: 5 of the 73 commits that added one (6.8%) carry an agent signature, against 9.0% of all window commits in the same repositories. Exposure thus came through committed template and environment files during the event, not through agent-signed commits or missing ignore rules.

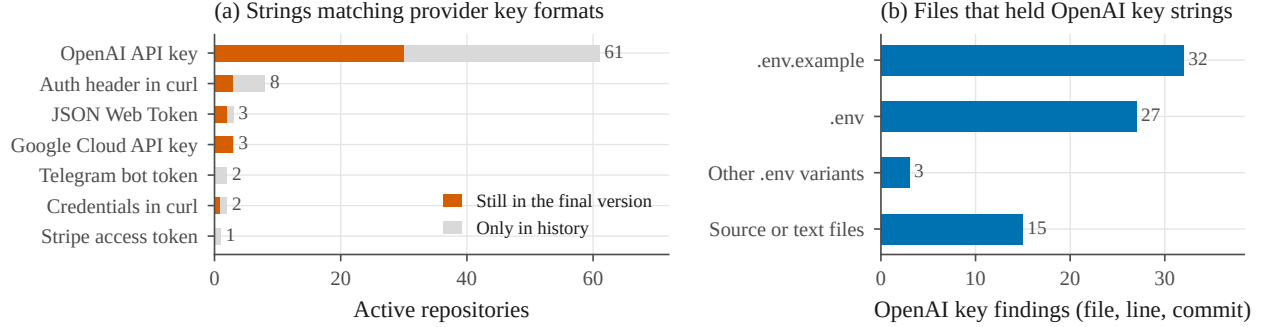


Figure 6: Credential exposure. (a) Active repositories with strings that match provider-specific key formats, split by whether the string is still present in the final version. (b) Files that held the OpenAI key strings; a finding is one file, line and commit, so a key can be counted more than once.

6 Discussion

6.1 Denominators decide rates

The same event yields an activity rate of 29.0%, 33.0% or 48.2%, depending on whether the repositories closed by the organisers and the batch created the evening before are counted. Taken at face value, all repositories suggest that the date of registration mattered a great deal ($\chi^2(3) = 608.2$, $V = 0.41$), whereas among open repositories outside the batch the effect is small ($V = 0.08$). We do not know why the organisers archived repositories in bulk or created the batch; withdrawn or duplicate registrations and the allocation of limited seats, which the news reports describe [51, 52], are possible reasons. The lesson for research is general. Organiser-created repositories record the organisers’ actions as well as the teams’, so archive times, creation bursts and template commits have to be reconstructed before any rate is read as participation. Datasets built from project listings face the opposite problem, since they never contain the empty repositories [13, 14].

6.2 What traces reveal when the tool is known

Every team was told to use Codex, yet the trace heuristics used in adoption studies detected Codex in 46.4% of active repositories, and in 69.6% of those that named Codex in their README. If all teams complied, a trace-based study of this event would have missed about half of the use of the required tool. The loss is tool-specific. Claude Code adds a co-author trailer to its commits by default [21], and among the repositories that named each tool, signatures revealed Claude Code in 58.0% of cases and Codex in 7.7%. Adoption estimates that rely on signatures [2, 3] will therefore favour tools that sign by default. This is the partial observability that Robbes et al. describe [4]; a known required tool and README self-reports make it possible to size it. The larger size of agent-signed commits agrees with their finding that agent-assisted commits are larger [3], here within the same repositories and in a setting where the tool was not chosen freely. The Claude Code traces also give a lower bound on the use of a tool that the rules did not require, in 24.9% of active repositories. The rule named Codex; whether it excluded other tools is not stated in the sources we read, so we do not treat these traces as breaches.

6.3 Artefacts, language and exposure

Test files, long READMEs with run instructions and, in a third of repositories, Docker files were produced within five hours. One explanation, which our data cannot test, is that agents make such artefacts cheap to produce; in many real agent sessions the agent writes virtually all committed code [1]. Whether the tests run or pass is unknown, and the quality risks of agent-written code are documented elsewhere [7, 49, 50]. The documentation was written mostly in Russian, and Kazakh was rare even in the track built around Kazakh audio. Language models support Kazakh less well than high-resource languages [58], which may push teams that work through agents towards Russian or English, but the audience that teams wrote for may matter as much, and our data cannot separate the two. The exposed keys followed a route that simple measures would have closed: they sat in templates that are meant to be committed, and neither agent-signed commits nor missing ignore rules explain them. Repository templates that ship a `.gitignore` and a placeholder-only `.env.example`, organisation-level secret scanning with push protection, and short-lived keys issued for the event are consistent with earlier recommendations [41, 44, 45].

6.4 Implications

For organisers, creating repositories after check-in would turn the organisation into a record of participants rather than of registrations, and the measures above would reduce the exposure of keys issued for an event. For researchers who measure agent use from repositories, our results suggest reporting how the population was formed, combining several kinds of trace for each tool, and stating what each trace can recover. Where possible, trace-based estimates should be calibrated against a reference such as self-reports or a known required tool. For studies of organiser-created repositories in particular, archive and creation events, template commits made by several organiser accounts and commits outside the default branch all have to be checked before counting.

7 Threats to validity

We organise the threats following the empirical standards for repository mining [59].

Construct validity. A repository stands for a registration, and author identities overstate the number of people, so we make no claims about individuals. Commit counts measure activity, not effort or quality, and test files show an intention to test, not working tests. Agent traces are partial by design, and the README self-report is an imperfect reference: a README may name a tool that was barely used, may be written by the agent itself, or may omit a tool that was used. We therefore report what each channel recovers and treat none of them as a measure of use. The README language rule is a letter heuristic that misses Kazakh typed with Russian letters; its accuracy against blind hand labels is [pending: being measured]. Credential findings match key formats, were not tested, and depend on a placeholder filter that was not validated against hand labels.

Internal validity and reliability. The event window is inferred rather than taken from a schedule, but it is supported by the archive times, and shifting it changes the number of active repositories little (Table 2). Features are measured at the final state, which in some repositories includes commits from before the event or after the deadline; leaving those repositories out barely changes the shares. Reading bulk archiving from last-update times is an inference, consistent with the absence of any commit in those repositories during the event. The associations with AGENTS.md, commit size and .gitignore are observational, and the self-report and commit-size analyses were added after the analysis plan. Track labels were produced and checked by one person. The collection, analysis and figure code, the analysis plan with its deviations and the derived data are released, and the commit at which each clone ended is listed so that the data can be collected again.

External validity. We study one in-person event in one country, with one required agent and a five-hour window. Other events differ in duration, rules, tools and participants, the default behaviour of agents changes between versions, and the share of empty repositories depends on how an organiser provisions them.

8 Conclusion

A hackathon at which the organisers created the repositories and fixed the tool allows repository measures to be checked against what was known in advance. At HackAlem AI, the organisers’ own actions moved the activity rate by 19.2 percentage points. The required agent left a trace in fewer than half of the active repositories and in fewer than three quarters of those that named it, while a non-required agent that signs its commits by default appeared in a quarter. Keys became public mainly through committed template and environment files. Studies of AI coding agents that start from repositories should therefore report how their population was formed and what each trace can recover. After the awards, we will test the pre-registered hypotheses about the jury’s decisions and compare these repositories with hackathon repositories from before coding agents [15].

Declarations

Ethics. All data come from public repositories and public web pages, the study involved no interaction with participants, and we accessed no private system. Author names and e-mail addresses were matched against agent patterns in memory and never stored; distinct authors were counted through a salted hash whose salt was never saved. Secret values were redacted by the scanner, never stored or printed by our code and never tested, and credentials are reported only in aggregate. We informed the organisers privately about the repositories containing credentials before publication [pending: confirm after contacting the organisers]. The author is not affiliated with the organisers, and this paper reports no official results of the event. [pending: Nazarbayev University ethics review or exemption statement]

Data and code availability. The pseudonymised dataset, the pipeline, the analysis plan and its deviations are available at <https://github.com/tbayan/HackalemaI-Repo-Data-Mining> [pending: archived release with DOI]. The released data use random identifiers; because the source repositories are public, this is pseudonymisation and not anonymisation. The analysis of the jury’s decisions was pre-registered before any finalist or winner was announced [60] [pending: OSF registration link].

Use of AI tools. The collection and analysis scripts, the figures and drafts of this text were produced with Claude Code (Anthropic), an AI coding agent using the Claude Opus 5.5 model, under the author’s direction. No language model labelled or classified the data; all measures come from the deterministic scripts described in Section 4. Two rounds of internal review were run with separate instances of the same agent, and the issues they raised and the changes made are released with the data. The author defined the research questions, definitions and analysis plan, checked the numbers against the data and the references against their sources, and takes full responsibility for the content. [pending: other AI tools, if any]

References

- [1] Joachim Baumann, Vishakh Padmakumar, Xiang Li, John Yang, Diyi Yang, and Sanmi Koyejo. SWE-chat: Coding Agent Interactions From Real Users in the Wild. arXiv preprint arXiv:2604.20779, 2026. URL <https://arxiv.org/abs/2604.20779>.
- [2] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. arXiv preprint arXiv:2507.15003, 2025. URL <https://arxiv.org/abs/2507.15003>.
- [3] Romain Robbes, Théo Matricon, Thomas Degueule, Andre Hora, and Stefano Zacchiroli. Agentic Much? Adoption of Coding Agents on GitHub. *ACM Transactions on Software Engineering and Methodology*, June 2026. doi:10.1145/3822180.
- [4] Romain Robbes, Théo Matricon, Thomas Degueule, Andre Hora, and Stefano Zacchiroli. Promises, Perils, and (Timely) Heuristics for Mining Coding Agent Activity. In *Proceedings of the 23rd International Conference on Mining Software Repositories*, MSR ’26, page 496–507. ACM, April 2026. doi:10.1145/3793302.3793375.
- [5] Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. Agent READMEs: An Empirical Study of Context Files for Agentic Coding. *ACM Transactions on Software Engineering and Methodology*, September 2026. doi:10.1145/3840295.
- [6] Shyam Agarwal, Hao He, and Bogdan Vasilescu. AI IDEs or Autonomous Agents? Measuring the Impact of Coding Agents on Software Development. In *Proceedings of the 23rd International Conference on Mining Software Repositories*, MSR ’26, page 857–862. ACM, April 2026. doi:10.1145/3793302.3793589.
- [7] Hao He, Courtney Miller, Shyam Agarwal, Christian Kästner, and Bogdan Vasilescu. Speed at the Cost of Quality: How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects. In *Proceedings of the 23rd International Conference on Mining Software Repositories*, MSR ’26, page 181–193. ACM, April 2026. doi:10.1145/3793302.3793349.
- [8] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories*, ICSE ’14, page 92–101. ACM, May 2014. doi:10.1145/2597073.2597074.
- [9] Sebastian Baltes and Paul Ralph. Sampling in software engineering research: a critical review and guidelines. *Empirical Software Engineering*, 27(4), April 2022. doi:10.1007/s10664-021-10072-8.
- [10] Connie W. Chau and Elizabeth M. Gerber. On Hackathons: A Multidisciplinary Literature Review. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI ’23, page 1–21. ACM, April 2023. doi:10.1145/3544548.3581234.
- [11] Alexander Nolte, Irene-Angelica Chounta, and James D. Herbsleb. What Happens to All These Hackathon Projects? Identifying Factors to Promote Hackathon Project Continuation. *Proceedings of the ACM on Human-Computer Interaction*, 4(CSCW2):1–26, October 2020. doi:10.1145/3415216.
- [12] Lukas McIntosh and Caroline D. Hardin. Do Hackathon Projects Change the World? An Empirical Analysis of GitHub Repositories. In *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education*, SIGCSE ’21, page 879–885. ACM, March 2021. doi:10.1145/3408877.3432435.

- [13] Ahmed Imam, Tapajit Dey, Alexander Nolte, Audris Mockus, and James D. Herbsleb. The Secret Life of Hackathon Code Where does it come from and where does it go? In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, page 68–79. IEEE, May 2021. doi:[10.1109/msr52588.2021.00020](https://doi.org/10.1109/msr52588.2021.00020).
- [14] Ahmed Samir Imam Mahmoud, Tapajit Dey, Alexander Nolte, Audris Mockus, and James D. Herbsleb. One-off events? An empirical study of hackathon code creation and reuse. *Empirical Software Engineering*, 27(7), September 2022. doi:[10.1007/s10664-022-10201-x](https://doi.org/10.1007/s10664-022-10201-x).
- [15] Sjoerd Halmans, Lavinia Paganini, Alexander Serebrenik, and Alexander Nolte. HackRep: A Large-Scale Dataset of GitHub Hackathon Projects. In *Proceedings of the 23rd International Conference on Mining Software Repositories, MSR '26*, page 579–583. ACM, April 2026. doi:[10.1145/3793302.3793305](https://doi.org/10.1145/3793302.3793305).
- [16] Wangyiyao Zhou, Alexander Serebrenik, and Alexander Nolte. Quick Build, Careful Check? Generative AI Use in Hackathons. In *Proceedings of the 2026 International Conference on Game Jams Hackathons and Game Creation Events, ICGJ 2026*, page 23–27. ACM, August 2026. doi:[10.1145/3833089.3833091](https://doi.org/10.1145/3833089.3833091).
- [17] Kiev Gama, Filipe Calegario, Victoria Jackson, Alexander Nolte, Luiz Moraes, and Vinicius Cardoso Garcia. “Can you feel the vibes?”: An exploration of novice programmer engagement with vibe coding. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering, ICSE-SEET '26*, page 485–496. ACM, April 2026. doi:[10.1145/3786580.3786992](https://doi.org/10.1145/3786580.3786992).
- [18] Ashley J. Chen, Yijia Cao, Minghao Shao, Ramesh Karri, and Muhammad Shafique. Code for All: Educational Applications of the “Vibe Coding” Hackathon in Programming Education across All Skill Levels. arXiv preprint arXiv:2604.22747, 2026. URL <https://arxiv.org/abs/2604.22747>.
- [19] Ramteja Sajja, Carlos Erazo Ramirez, Zhouyayan Li, Bekir Z. Demiray, Yusuf Sermet, and Ibrahim Demir. Integrating Generative AI in Hackathons: Opportunities, Challenges, and Educational Implications. *Big Data and Cognitive Computing*, 8(12):188, December 2024. doi:[10.3390/bdcc8120188](https://doi.org/10.3390/bdcc8120188).
- [20] HackAlem AI. HackAlem AI — Astana, 23.09.2026, 2026. URL <https://hackalem.ai/>. Official event website. Accessed 24 September 2026.
- [21] Anthropic. Claude Code settings reference: git and attribution, 2026. URL <https://code.claude.com/docs/en/settings-reference>. States that commits get a git trailer such as Co-Authored-By by default. Accessed 27 September 2026.
- [22] Christian Bird, Peter C. Rigby, Earl T. Barr, David J. Hamilton, Daniel M. German, and Prem Devanbu. The promises and perils of mining git. In *2009 6th IEEE International Working Conference on Mining Software Repositories*, page 1–10. IEEE, May 2009. doi:[10.1109/msr.2009.5069475](https://doi.org/10.1109/msr.2009.5069475).
- [23] Nicole Hoess, Carlos Paradis, Rick Kazman, and Wolfgang Mauerer. Does the Tool Matter? Exploring Some Causes of Threats to Validity in Mining Software Repositories. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, page 645–656. IEEE, March 2025. doi:[10.1109/saner64311.2025.00067](https://doi.org/10.1109/saner64311.2025.00067).
- [24] Rosalia Tufano, Antonio Mastropaolo, Federica Pepe, Ozren Dabic, Massimiliano Di Penta, and Gabriele Bavota. Unveiling ChatGPT’s Usage in Open Source Projects: A Mining-based Study. In *Proceedings of the 21st International Conference on Mining Software Repositories, MSR '24*, page 571–583. ACM, April 2024. doi:[10.1145/3643991.3644918](https://doi.org/10.1145/3643991.3644918).
- [25] OpenAI. Custom instructions with AGENTS.md (Codex documentation), 2026. URL <https://learn.chatgpt.com/docs/agent-configuration/agents-md>. States that Codex reads AGENTS.md files before doing any work. Accessed 27 September 2026.
- [26] Thibaud Gloaguen, Niels Mündler, Mark Müller, Veselin Raychev, and Martin Vechev. Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? arXiv preprint arXiv:2602.11988, 2026. URL <https://arxiv.org/abs/2602.11988>.
- [27] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-world GitHub Issues? In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html. Preprint: doi:[10.48550/arXiv.2310.06770](https://doi.org/10.48550/arXiv.2310.06770).
- [28] John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37, NeurIPS 2024*, page 50528–50652. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024. doi:[10.52202/079017-1601](https://doi.org/10.52202/079017-1601).

- [29] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv preprint arXiv:2302.06590, 2023. URL <https://arxiv.org/abs/2302.06590>.
- [30] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. Measuring GitHub Copilot’s Impact on Productivity. *Communications of the ACM*, 67(3):54–63, February 2024. doi:[10.1145/3633453](https://doi.org/10.1145/3633453).
- [31] Shraddha Barke, Michael B. James, and Nadia Polikarpova. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):85–111, April 2023. doi:[10.1145/3586030](https://doi.org/10.1145/3586030).
- [32] Jenny T. Liang, Chenyang Yang, and Brad A. Myers. A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ICSE ’24, page 1–13. ACM, February 2024. doi:[10.1145/3597503.3608128](https://doi.org/10.1145/3597503.3608128).
- [33] Muhammad Waseem, Aakash Ahmad, Kai-Kristian Kemell, Jussi Rasku, Sami Lahti, Kalle Mäkelä, and Pekka Abrahamsson. Vibe Coding in Practice: Flow, Technical Debt, and Guidelines for Sustainable Use. arXiv preprint arXiv:2512.11922, 2025. URL <https://arxiv.org/abs/2512.11922>.
- [34] James Prather, Paul Denny, Juho Leinonen, Brett A. Becker, Ibrahim Albluwi, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Andrew Luxton-Reilly, Stephen MacNeil, Andrew Petersen, Raymond Pettit, Brent N. Reeves, and Jaromir Savelka. The Robots Are Here: Navigating the Generative AI Revolution in Computing Education. In *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education*, ITiCSE 2023, page 108–159. ACM, December 2023. doi:[10.1145/3623762.3633499](https://doi.org/10.1145/3623762.3633499).
- [35] Paul Denny, James Prather, Brett A. Becker, James Finnie-Ansley, Arto Hellas, Juho Leinonen, Andrew Luxton-Reilly, Brent N. Reeves, Eddie Antonio Santos, and Sami Sarsa. Computing Education in the Era of Generative AI. *Communications of the ACM*, 67(2):56–67, January 2024. doi:[10.1145/3624720](https://doi.org/10.1145/3624720).
- [36] Maëlick Claes, Mika V. Mäntylä, Miikka Kuutila, and Bram Adams. Do programmers work at night or during the weekend? In *Proceedings of the 40th International Conference on Software Engineering*, ICSE ’18, page 705–715. ACM, May 2018. doi:[10.1145/3180155.3180193](https://doi.org/10.1145/3180155.3180193).
- [37] Stephen H. Edwards, Jason Snyder, Manuel A. Pérez-Quinones, Anthony Allevato, Dongkwan Kim, and Betsy Tretola. Comparing effective and ineffective behaviors of student programmers. In *Proceedings of the fifth international workshop on Computing education research workshop*, ICER ’09, page 3–14. ACM, August 2009. doi:[10.1145/1584322.1584325](https://doi.org/10.1145/1584322.1584325).
- [38] Ayaan M. Kazerouni, Stephen H. Edwards, and Clifford A. Shaffer. Quantifying Incremental Development Practices and Their Relationship to Procrastination. In *Proceedings of the 2017 ACM Conference on International Computing Education Research*, ICER ’17, page 191–199. ACM, August 2017. doi:[10.1145/3105726.3106180](https://doi.org/10.1145/3105726.3106180).
- [39] Miikka Kuutila, Mika Mäntylä, Umar Farooq, and Maëlick Claes. Time pressure in software engineering: A systematic review. *Information and Software Technology*, 121:106257, May 2020. doi:[10.1016/j.infsof.2020.106257](https://doi.org/10.1016/j.infsof.2020.106257).
- [40] Zeinabsadat Saghi, Thomas Zimmermann, and Souti Chattopadhyay. Code Today, Deadline Tomorrow: Procrastination Among Software Developers. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, page 1204–1216. IEEE, April 2025. doi:[10.1109/icse55347.2025.00198](https://doi.org/10.1109/icse55347.2025.00198).
- [41] Michael Meli, Matthew R. McNiece, and Bradley Reaves. How Bad Can It Get? Characterizing Secret Leakage in Public GitHub Repositories. In *Proceedings 2019 Network and Distributed System Security Symposium*, NDSS 2019. Internet Society, 2019. doi:[10.14722/ndss.2019.23418](https://doi.org/10.14722/ndss.2019.23418).
- [42] Vibha Singhal Sinha, Diptikalyan Saha, Pankaj Dhoolia, Rohan Padhye, and Senthil Mani. Detecting and Mitigating Secret-Key Leaks in Source Code Repositories. In *2015 IEEE/ACM 12th Working Conference on Mining Software Repositories*, page 396–400. IEEE, May 2015. doi:[10.1109/msr.2015.48](https://doi.org/10.1109/msr.2015.48).
- [43] Setu Kumar Basak, Lorenzo Neil, Bradley Reaves, and Laurie Williams. SecretBench: A Dataset of Software Secrets. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, page 347–351. IEEE, May 2023. doi:[10.1109/msr59073.2023.00053](https://doi.org/10.1109/msr59073.2023.00053).
- [44] Setu Kumar Basak, Jamison Cox, Bradley Reaves, and Laurie Williams. A Comparative Study of Software Secrets Reporting by Secret Detection Tools. In *2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, page 1–12. IEEE, October 2023. doi:[10.1109/esem56168.2023.10304853](https://doi.org/10.1109/esem56168.2023.10304853).
- [45] Alexander Krause, Jan H. Klemmer, Nicolas Huaman, Dominik Wermke, Yasemin Acar, and Sascha Fahl. Pushed by Accident: A Mixed-Methods Study on Strategies of Handling Secret Information in Source Code Repositories. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 2527–2544, Anaheim, CA, August 2023.

- USENIX Association. ISBN 978-1-939133-37-3. URL <https://www.usenix.org/conference/usenixsecurity23/presentation/krause>. Preprint: doi:10.48550/arXiv.2211.06213.
- [46] Yizhan Huang, Yichen Li, Weibin Wu, Jianping Zhang, and Michael R. Lyu. Your Code Secret Belongs to Me: Neural Code Completion Tools Can Memorize Hard-Coded Credentials. *Proceedings of the ACM on Software Engineering*, 1(FSE):2515–2537, July 2024. doi:10.1145/3660818.
- [47] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. Do Users Write More Insecure Code with AI Assistants? In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS '23*, page 2785–2799. ACM, November 2023. doi:10.1145/3576915.3623157.
- [48] Pinran Gao, Lingxiang Wang, Yi Liu, Kunpeng Liu, Fan Yang, and Ying Zhang. Mind your key: An Empirical Study of LLM API Credential Leakage in iOS Apps. arXiv preprint arXiv:2606.12212, 2026. URL <https://arxiv.org/abs/2606.12212>.
- [49] Songwen Zhao, Danqing Wang, Kexun Zhang, Jiaxuan Luo, Zhuo Li, and Lei Li. Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks. arXiv preprint arXiv:2512.03262, 2025. URL <https://arxiv.org/abs/2512.03262>.
- [50] Junquan Deng, Zhiyu Fan, and Ruijie Meng. Understanding the (In)Security of Vibe-Coded Applications. arXiv preprint arXiv:2606.23130, 2026. URL <https://arxiv.org/abs/2606.23130>.
- [51] Qumash.kz. Five hours in the queue: HackAlem AI participants in Astana complained about the organisation, September 2026. URL <https://qumash.kz/news/pyat-chasov-v-ocheredi-uchastniki-hackalem-ai-v-astane-pozhalovalis-na-organizatsiyu/>. In Russian. Accessed 24 September 2026.
- [52] Ulys Media. “Survival simulator”: participants criticised HackAlem AI in Astana, September 2026. URL <https://ulysmedia.kz/news/81431-simuliator-vyzhivaniia-uchastniki-raskritikovali-hackalem-ai-v-astane/>. In Russian. Accessed 24 September 2026.
- [53] The Astana Times. Astana stages massive AI hackathon in Guinness World Records bid, September 2026. URL <https://astanatimes.com/2026/09/astana-stages-massive-ai-hackathon-in-guinness-world-records-bid/>. Accessed 24 September 2026.
- [54] Kazinform. What is built in five hours: how the largest AI hackathon in Astana is run, September 2026. URL <https://www.inform.kz/ru/chto-sozdayut-za-pyat-chasov-kak-prohodit-krupneyshiy-ai-hakaton-v-astane-72e380f9>. In Russian. Accessed 24 September 2026.
- [55] Gede Artha Azriadi Prana, Christoph Treude, Ferdian Thung, Thushari Atapattu, and David Lo. Categorizing the Content of GitHub README Files. *Empirical Software Engineering*, 24(3):1296–1327, October 2018. doi:10.1007/s10664-018-9660-3.
- [56] Gitleaks contributors. Gitleaks: a tool for detecting secrets in git repositories, version 8.30.1, 2026. URL <https://github.com/gitleaks/gitleaks>. Release v8.30.1, 21 March 2026. Accessed 24 September 2026.
- [57] Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, November 1987. doi:10.1016/0377-0427(87)90125-7.
- [58] Mukhammed Togmanov, Nurdaulet Mukhituly, Diana Turmakhan, Jonibek Mansurov, Maiya Goloburda, Akhmed Sakip, Zhuohan Xie, Yuxia Wang, Bekassyl Syzdykov, Nurkhan Laiyk, Alham Fikri Aji, Ekaterina Kochmar, Preslav Nakov, and Fajri Koto. KazMMLU: Evaluating Language Models on Kazakh, Russian, and Regional Knowledge of Kazakhstan. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, page 14403–14416. Association for Computational Linguistics, 2025. doi:10.18653/v1/2025.acl-long.701.
- [59] Paul Ralph, Nauman bin Ali, Sebastian Baltes, Domenico Bianculli, Jessica Diaz, Yvonne Dittrich, Neil Ernst, Michael Felderer, Robert Feldt, Antonio Filieri, Breno Bernard Nicolau de França, Carlo Alberto Furia, Greg Gay, Nicolas Gold, Daniel Graziotin, Pinjia He, Rashina Hoda, Natalia Juristo, Barbara Kitchenham, Valentina Lenarduzzi, Jorge Martínez, Jorge Melegati, Daniel Mendez, Tim Menzies, Jefferson Moller, Dietmar Pfahl, Romain Robbes, Daniel Russo, Nyyti Saarimäki, Federica Sarro, Davide Taibi, Janet Siegmund, Diomidis Spinellis, Mirosław Staron, Klaas Stol, Margaret-Anne Storey, Davide Taibi, Damian Tamburri, Marco Torchiano, Christoph Treude, Burak Turhan, Xiaofeng Wang, and Sira Vegas. Empirical Standards for Software Engineering Research. arXiv preprint arXiv:2010.03525, 2020. URL <https://arxiv.org/abs/2010.03525>.
- [60] Olmo R. Van den Akker, Sara Weston, Lorne Campbell, Bill Chopik, Rodica Damian, Pamela Davis-Kean, Andrew Hall, Jessica Kosie, Elliott Kruse, Jerome Olsen, Stuart Ritchie, KD Valentine, Anna Van ’t Veer, and Marjan Bakker. Preregistration of secondary data analysis: A template and tutorial. *Meta-Psychology*, 5, November 2021. doi:10.15626/mp.2020.2625.